

Audience: LEWG, WG21

Document number: P2332R0

Date: 2021-03-07

Project: Establishing std::hive as replacement name for the proposed std::colony container

Reply-to: Matthew Bentley <mattreecebentley@gmail.com>, Ville Voutilainen <ville.voutilainen@gmail.com>, Gašper Ažman <gasper.azman@gmail.com>

ESTABLISHING STD::HIVE AS REPLACEMENT NAME FOR THE PROPOSED STD::COLONY CONTAINER

Table of Contents

- I. [Overview](#)
- II. [Reasoning for not selecting an existing name from similar but not identical structures](#)
- III. [Closing comment](#)
- IV. Appendices:
 - A. [Ben Craig's summary of names](#)
 - B. [Expanded reasoning for not selecting existing names with examples](#)
 - C. [Nathan Myers' short list of naming guidelines](#)

I. Overview

This document is to signify the change the name of the proposed std::colony container (D0447R12) to std::hive. It serves as both a small summary of the conversations had around the topic, a reference point for anyone in the future wishing to critique or understand said naming, and possibly as a small aid to future naming discussions. The reasoning presented below does not necessarily represent the general position, but the position of the authors.

The original name, colony, while seen as non-problematic by the author and users, was seen as problematic by many members of LEWG. Leaving aside subjective feelings of 'not liking' the name,

which have no relevance in discussion, the objective critiques were as follows:

- the name has too many meanings
- the name is unfamiliar and not in common usage

The second point is more of an advantage than a disadvantage and the reasoning for this will be explained in the following section. The first is somewhat valid, although 'colony' as a general term refers to a collection of individuals in a greater group, where the individual members come and go frequently. This is as true of bacterial colonies as it is of ant colonies, termite colonies or human colonies.

The author's reasoning was that the name should ideally describe the structure of the container in an intuitive manner, and the container is itself (a) a collection of memory blocks with a growth factor rather than a single contiguous block (in order to prevent reallocation upon insertion) (b) elements within those memory blocks which do not reallocate within the lifetime of the element (barring some uncommon operations like `shrink_to_fit`) (c) an $O(1)$ erased-element-skipping mechanism (to prevent reallocation upon insertion) and (d) an erased-element-reuse mechanism. It has been noted that the specific erased-element-skipping-mechanism is not itself a useful naming influence since this could differ from implementation-to-implementation.

The term 'colony' rather neatly encapsulated this structural meaning when viewed as as human colony with 1. the container itself being the colony, containing 2. the memory blocks being houses, containing 3. rooms ie. element memory spaces from which 4. people ie. elements come and go. This metaphor works as well for ant colonies with tunnels instead of houses, but breaks down by the time you reach bacterial colonies and the like, and the author acknowledges this.

'Hive', by comparison only really has one strong primary association, that of the beehive, wherein, [at least in a man-made one](#), there are (a) slabs containing (b) the hexagonal 'cells', from which (c) bees come and go frequently. This also maps reasonably neatly onto the container, with slabs being memory blocks and cells being element memory spaces. The only downside to this metaphor is that bees don't live in the cells, so the 'stable location' aspect of the original metaphor is lost. 'stable location' refers to the fact that elements in this container do not reallocate when other elements are inserted or erased.

However this was seen by many members of LEWG as a good-enough sacrifice to make in order to obtain a term which does not have too many alternative meanings or possible variants, thereby crystalising the metaphorical meaning somewhat. An additional bonus from the author's perspective is that this container outperforms both non-standard-library custom containers and standard-library containers in tests where (a) stable element locations are essential and (b) the ratio of erasure/insertion to iteration is relatively high eg. erasing/inserting 1% of all elements per 60 iterations.

So the frequency of arrival and departure of a beehive (and general 'busy-ness') naturally captures this aspect, and in doing so gives a hint as to its optimum usage when being compared to other possible custom solutions. In addition the container is unordered in terms of insertion location and the somewhat chaotic nature of a hive is seen to reflect this to an extent. Lastly the name is short,

succinct, and particularly for a template container which must inevitably trail screeds of angle-brackets and double-semi-colons in its instantiation and the instantiation of its iterators, brevity helps code clarity somewhat.

No metaphor is perfect and no name will satisfy everyone, but the general feeling was that there was a greater consensus toward `hive` than any of the other names proposed. The author is comfortable with the name and will amiably support/allow it.

II. Reasoning for not selecting an existing name from similar but not identical structures

While many names were proposed (see [Appendix A](#) for Ben Craig's almost-complete-summary of some of the proposed names and his take on the benefits/detriments) many of the names were for pre-existing structures which have some similarity to the proposed colony container. `Colony` is itself an outgrowth from what is typically called a bucket array in gaming circles. Here we'll discuss the reasons why these names were not selected.

The main reason was that none of these names (`bucket`, `pool`, `bag`, etc) adequately communicate the structure of the container or give a hint as to its behaviour or usage. At best they all signify an unordered collection of elements, which is as true of `unordered_map` and other containers (in fact, the term 'bag' is historically synonymous with multisets). 'Pool' in particular communicates a pool of homogenous resources, rather than a collective of individuals - this is sufficient for an allocator, but not for this container. The other reasons are summarised as follows (and explored in more depth in [Appendix B](#)):

1. Those structures have subtle and unsubtle differences which would impede understanding of this structure if the user had experience with a structure with the same name. eg. pools do not tend to have iteration or skipping mechanisms, and a user might not understand that there is an additional memory cost incurred by a having a skipping mechanism.
2. This structure provides more functionality than those structures do, and a user familiar with the name may not explore this structure adequately, thereby possibly not finding better ways of performing tasks.
3. Since those structures are all non-standard, each custom version will be different from location to location. Therefore, a user might become disappointed that an aspect of their container of the same name was not reflected in the standardised version. They would likely ignore the additional functionality that does exist in this version and ignore the container without exploring what other areas it might be better suited to than their existing containers.
4. Choosing between those names, at least on an objective basis, is an impossible task, since each person would be more familiar with one than the other, and it would come down to familial preference.
5. The exact combination of aspects which each of those containers have, is very specific. A pool doesn't have an iteration mechanism, a bucket array doesn't have a reuse mechanism (typically) or a growth factor for memory blocks, or an $O(1)$ skipping mechanism, other containers have

singular memory blocks with an iteration mechanism. None of them have the combination of aspects which this container has (as described in the overview). Having one name that doesn't match any of those structures helps users to distinguish it and increases clarity.

6. Giving it a new name mitigates against people presuming, probably incorrectly, that they already understand the container based on their prior experience.

III. Closing comment

It should be noted that all names become associational over time regardless of their original intention or structure ie. we do not resolve back to the latin root words for most english words when we think of them, we merely associate. It is expected that regardless of the name selected, it will become associated with the programming structure, when used in the context of programming, rather than people reflecting on its real-world counterpart. However, it can help to have an appropriately reflective word, particularly for beginners and for those who're coming to the container for the first time. None of the existing container names in the standard library are perfect metaphors but they give a general indication as to structure and as such, suffice.

While there may be some who negatively associate with the word 'hive', there're also those who negatively associated with the word 'colony'. But, since all meaning is contextual, like the terms list, map and vector, the association would shift to being in connection with the container once safely ensconced within the context of C++.

IV. Appendices

Appendix A - Ben Craig's summary of names

Matt note: I agree with some of this. The main name missing here is pool, but we've covered that in the above sections. I've edited out a couple of technical inaccuracies and annotated a couple of places.

"My opinions on some of the names and name pieces I've seen (and some that I've suggested), bucketed in WF, N, WA, and SA.

SA:

- **population:** reused heavily in statistics. A population also has some sense of agency, where a container generally does not. I would not guess that something called `std::population` is a container. The name-by-analogy doesn't help me much here. The analogies I've heard apply equally well to other containers.
- **colony:** Similar to population, a colony has some sense of agency. I wouldn't guess that this is a container, and the analogy doesn't help me much here. There are political / cultural issues with the name too.
- **_deque, *_allocator:** The implementation has some similarities to deques and allocators, but the user interface of deque and allocator bears little to no resemblance to `std::colony`. (Matt: deques can be implemented as linked lists, but are more commonly implemented as collections of memory blocks. That is the only similarity. See appendices in the colony paper for more info re: allocators)

WA:

- **lot, garage, park:** These things don't have agency, which is an improvement, but none of these names feel like a container. The analogy is more helpful to me, but it still has to be explained.
- **parcel:** I associated this with the mailing definition (e.g. parcel post). As a result, I didn't really get the analogy here even when it was explained. The "group of things" definition applies equally well to other containers.
- **_sparse:** The implications on how much storage is used would be opposite what is used for matrices. A sparse matrix consumes less memory than a dense matrix.
- **_store, _storage:** All containers store things, so this doesn't really tell me anything. If we are going to add terms that don't help us decide between containers, then I'd rather they be more unique, like **hive**.
- ***_bundle, bag:** Generic container word that has been used elsewhere with different meaning.
- ***_sack:** probably too close to bag?
- ***_deposit, *_dump:** not things I would associate with a container, or iterating. Maybe that's because I don't go looking for old car parts frequently...
- ***_cluster:** Other languages use the term cluster like we use `struct`. Clusters seem more heterogenous to me. The word does sound generically containerish though.

N:

- **hive:** hive feels more container like, and doesn't have agency. I could live with this name. I would prefer it say more about when to use the container. The name lacks a lot of positives, but it doesn't have any huge negatives either.
- **hotel:** I like hive a little better than this one, though I'm not sure why. What I said about hive also applies to hotel.

- `stable_*`: One of the big reasons to use this container is that references aren't invalidated. However, `stable_` has meaning in the standard for algorithms (eg. `sort`). We also have a `_lot_` of other containers which provide reference stability, but don't mention it in their names.
- `bucket_` and `*_bucket`: buckets certainly feel containerish, but don't really convey much information beyond that. The term is somewhat overloaded thanks to the unordered associative containers. A big bonus for using bucket is that many of the informal descriptions of the container use the word bucket as well. A negative is that other containers in the wild likely use the word container, and may mislead people as to what this thing does.
- `bucket_array`: `*_array` in general feels like a trap due to the C and C++ expectations of what you can do with an array (contiguous storage, random access), but I think `bucket_array` is tolerable, largely because the container is often described as a bucket array.
- `skip_*`: This describes some of the implementation, and not how the container is used. I can live with it, but I'm not thrilled with it.

WF:

- `unordered_*`: I like this because it tells you when (not) to use the container. Also, attaching this prefix to some other term helps make the other term "feel" more like a container than it would otherwise. If I saw "`unordered_widget`" (as an example), I would suspect the thing is a container, where I don't think I would feel the same about "`widget`" on its own.
- `_list`: The iterators of `std::list` are bidirectional. You can splice full containers. You have reference stability. It isn't tuned for lookup. These are all points in favor of list. The main thing I don't like about `_list` is the implication that it's a linked list. However, other languages use "list" for their main growable array type, so I don't think that's a show stopper." (Matt: I think the majority of people seeing `_list` would assume it is some sort of linked list in the context of C++, as we already have `forward_list`)

(Matt: 'cote' was also mentioned as an alternative to `hive`, but is considered an unfamiliar (within and without of programming circles) and therefore not useful term, as the primary motivation for an appropriate name is to give a cognitive handle on the structure for beginners, whereas experienced users will merely associate the name directly. The need to explain the term to beginners takes away its benefits.)

Appendix B - Expanded reasoning for not selecting existing names with examples (Matt)

1. As an example, bucket arrays do not commonly have erased-element-memory-location-reuse mechanisms. Thereby, somebody who was using a bucket array would reasonably expect that a pointer to one element would never point to another element once that element was erased. Depending on how they had structured and used the bucket array, they might be using the skipfield bit/byte as a 'tombstone' ie. an indicator that the element is erased, as a way to

communicate to other structures which link to that element that the element is erased and should not be accessed anymore - rather than proactively going out and updating each of those structures. This is one common pattern in game engines. In this case the implementation would not be removing the memory blocks when they become empty, but delaying deallocation until a particular point.

This is of course not something which can be employed in a colony. The same action (checking for a 'tombstone') can almost be performed with the 'get_iterator_from_pointer' function - if the pointer points to an element which's been erased, that function will return end(). But due to the erased-element-reuse mechanism there is no guarantee given by the container that the element has not been erased and then replaced with another one via insert/emplace, in which case the function will still return a non-end() iterator, and the external caller would be none the wiser.

2. A bucket array might have insert, erase, and some form of iteration and access. It is unlikely to have much else, so making the assumption that colony might be similar, may lead others to not investigate further and find what this container is capable of. Sorting, whole-container splicing, move-insert, emplacement, range-erase/insert, all the other forms of insert and constructor, reserve, shrink_to_fit, assign, trim, optimal advance/prev/next/distance overloads, etc, etc, etc. The effect of this is two-fold. For starters the container is far more 'capable' and may provide new ways of thinking about how to perform certain tasks - but also the memory usage of the container code is higher. Someone looking for a bucket array might assume something much more sparse, and this would either lead to disappointment or pleasant surprise depending on priorities, but in either case the assumption is an inaccurate fit.
3. Since all of these various containers with similar properties to each other have always been non-std::, custom containers, and (until now) there has been little communication between fields about these types of containers, the likelihood that each users' custom implementation has situation-specific functionality which colony might not have, is high. Such a user might look at colony and dismiss it out of hand for not having feature Z, while ignoring the features A-Y which it does have and the custom implementation does not, and which might allow them to think about their problems in different, possibly more-optimal ways. Also, programmer-inertia being what it is, it's likely that such a user would be sticking with their own custom structure for now, regardless, so it's best if they're encouraged to come to this container with a fresh eye, in order to see what it might be best used for.
4. I have heard of pool-like structures referred to as 'free lists', despite the fact that that merely describes the reuse mechanism rather than the structure as a whole. If based on prior naming, the question therefore becomes simply a matter of preference, rather than importance or accuracy.
5. Similar structures I have heard about in high-energy physics simulation, in high-performance trading, in gaming, in allocation, tend to have one or two of these aspects, but not all four (and definitely not O(1) skipping). Some may use a fixed-size array, a boolean skipfield and a reuse mechanism, or like a bucket array, they may use a multiple-memory-block model without a growth factor or reuse and with a boolean skipfield. Some, like a pool, only have a multiple-memory-block allocation pattern, typically without a growth factor, and they reuse, but have no iteration. Having a new name helps us to be specific about the structure without causing confusion.
6. Overall it is better to have the name be new and not strongly associated with any existing containers in order to be taken 'as it is' and fulfill its potential in terms of usage. In addition there has already been a problem with communicating colony in that many, many people have tried to

tell me 'it's like X, but with Y' and have all been incorrect. In short, people already want to assume they understand something without putting the time and effort into investigating whether that is actually true. A new name does not entirely mitigate against this but helps.

Appendix C - Nathan Myers' short list of naming guidelines

"We like names that are

1. long enough; the object must be distinguished from other objects in its category
2. short enough; we all have many other uses for the space on any line of source code, and in our short-term memory, which counts syllables, so a big name is a big evil.
3. suggestive; while there is never enough room in a name for a full spec, we all need a hint sometimes.
4. accurate; an actively misleading name produces unbounded confusion: it misleads everywhere it appears.
5. direct; evoking extraneous associations burns attention.
6. precise: it should not equally well refer to objects the name does not.
7. distinct: it should not sound or look much like other names."